

ResQLink

Final End-to-End Plan - DBTHON 2026 + Patent-Oriented Architecture

Plan assessment based on the uploaded final plan and the DBTHON challenge/project documents already reviewed in this conversation.

Final rating: 9.1 / 10

Dimension	Rating	Assessment
DBTHON alignment	9.7/10	Directly addresses the required database innovation, novelty, prototype and measurable evaluation.
Technical depth	9.4/10	Strong use of PostgreSQL/PostGIS, transactions, locking, RLS, events, temporal state and benchmarking.
Novelty differentiation	8.7/10	Much stronger because the invention is centered on dependency-tracked incremental re-matching and contention-safe reservation.
Patent preparedness	7.8/10	Good technical direction, but patentability still depends on a real prior-art search and claim drafting by an IP professional.
Feasibility	8.2/10	Achievable, but Phase 2b-6 contain several interacting systems that must be kept under control.
Evaluation quality	9.5/10	Ablations, baselines, concurrency tests and scale tests make the claims measurable and defensible.
Demo strength	9.4/10	The affected-pool re-match demo with a live rows-evaluated counter is an excellent judge-facing moment.

Verdict: This is a strong final direction. Keep the core invention narrow and coherent; treat temporal confidence and hazard-aware accessibility as supporting mechanisms, not as separate inventions.

What is strong in the new plan

- It correctly moves away from a generic "nearest resource + urgency" application and gives the system one central technical mechanism.
- The core invention is tightly connected to a real database problem: avoiding unnecessary full re-computation when resource state changes.
- The plan makes concurrency a correctness property through leased reservations and transactional locking, instead of treating it as an afterthought.
- The evaluation directly measures database work saved, re-match latency, double-allocation rate, stale matches and optimality gap.
- The plan explicitly avoids overclaiming Sahana Eden and avoids treating "using PostGIS/Al/etc." as novelty by itself.

The five DBTHON novelty steps are now coherent

Step	New plan
Existing approach	Centralized inventory/request management and nearest-first dispatch; changes cause manual re-search or broad rescans.
Limitation	Full recomputation wastes DB work; stale availability creates failed dispatches; straight-line distance can be misleading; concurrent matchers can double-allocate scarce units.
Proposed approach	Event-driven matching that recomputes only dependent requests, using confidence-weighted state, hazard-aware reachability and leased atomic reservation.
Novel component	Pool-dependency index + dirty-set incremental re-matching + lease-based reservation inside PostgreSQL.
Measurable benefit	Fewer rows/pairs evaluated, zero double-allocation target, fewer stale matches, lower critical-request response time and lower p95 re-match latency at scale.

Remaining changes before Phase 3

- **Keep Phase 2b bounded.** Hazard-aware accessibility should first use polygon/zone reachability or a lightweight accessibility penalty. Do not build a full routing engine unless time remains.
- **Use outbox as the durability mechanism.** LISTEN/NOTIFY should only wake the worker; it should not be the source of truth because notifications are not a durable queue.
- **Be careful with the quantity ledger.** A trigger-maintained balance/cache may be safer for hot-path matching than repeatedly summing an insert-only ledger under concurrency.
- **Align ILP ground truth with the production objective.** Document the exact objective and constraints so the optimality gap is meaningful rather than arbitrary.
- **Do the prior-art search before public disclosure.** The plan is patent-oriented, not patent-approved. Novelty and inventive step must be checked claim-by-claim.
- **Protect feasibility.** Build the core end-to-end loop before adding stretch features such as offline reconciliation.

1. Core innovation: pick one, keep it coherent

Two documents proposed many ideas (confidence decay, pressure, accessibility, offline CRDTs, vectors, adaptive weights). Putting all of them in produces a feature list, which hurts both the novelty score and the patent. So the plan has one core claim and two supporting mechanisms.

Core invention: Dependency-tracked incremental re-matching with contention-safe reservation.

The database keeps a **pool-dependency index** that records which pending requests depend on which resource pools (zone x resource type x mobility class). When a resource changes (freed, moved, expired, or flagged stale), a trigger writes an event, and **only the requests in the affected pools** are re-matched. Allocation happens through atomic, leased reservations.

Supporting mechanism 1: Temporal confidence. Effective capacity is $qty \times confidence(age\ of\ last\ update)$, and location history is stored. This is a temporal-DB component.

Supporting mechanism 2: Hazard-aware accessibility. Matching uses mobility-specific reachability (boat vs. truck vs. on-foot) over a hazard layer (flooded polygons), not straight-line distance.

Stretch only (Phase 9, if time remains): offline edge reconciliation. The partition-tolerant reservation ledger is the strongest patent angle but also the riskiest to build. Don't let it endanger the core.

Challenge's 5-step novelty format

Step	Content
Existing approach	Centralised inventory/request systems (Sahana-style) and nearest-first dispatch. A change triggers manual re-search, or a full re-scan of pending requests.
Limitation	Full recomputation per change wastes DB work. Stale availability causes failed dispatches. Straight-line distance ignores flooded roads. Concurrent matchers double-allocate scarce units.
Proposed approach	Event-driven matching that recomputes only the requests dependent on a changed pool, using confidence-weighted state, hazard-aware reachability, and leased atomic reservation.
Novel component	Pool-dependency index + dirty-set incremental re-matching + lease-based reservation, all inside PostgreSQL.
Measurable benefit	% fewer rows/pairs evaluated vs. full re-scan, 0 double-allocations under concurrency, fewer stale-resource matches, lower critical-request response time, and $\geq X\%$ lower p95 re-match latency at 100k-1M rows.

Wording to fix: don't say Sahana "falls short on matching." Sahana does request/inventory matching. Say instead: *"existing platforms support request and inventory matching but not incremental, contention-aware re-matching under rapidly changing availability."*

2. Marks map: every rubric item to a concrete deliverable

#	Component (marks)	What you must show	Where it comes from
1	Problem & domain (4)	Target users (district control room, NGOs, shelter managers, boat owners), real event data (Chennai 2015 / Kerala 2018 / a recent cyclone), the visibility and coordination failure, and measured limitations of the existing approach	Phase 1 + a "naive baseline" run in Phase 7
2	DB design & modeling (5)	ER diagram, 3NF schema, constraints, lookup tables, PostGIS geometry (polygons for zones and hazards, points for others), temporal tables, ledger tables, decision records	Phases 1-2 (+2b)
3	DBMS implementation & depth (5)	Constraints, triggers, stored functions, transactions, FOR UPDATE SKIP LOCKED, GiST + partial + composite indexes, Row-Level Security, LISTEN/NOTIFY, pg_cron, EXPLAIN ANALYZE	Phases 2b-6

#	Component (marks)	What you must show	Where it comes from
4	Innovation (4)	The core mechanism, working end to end	Phase 4
5	Novelty (5)	The 5-step table above, a prior-art comparison table, and ablations proving each part matters	Phases 1, 7
6	SDG & impact (2)	SDG 11.5 (reduce disaster deaths and losses), 13.1 (climate-hazard resilience), 3, 17 (partnerships across agencies), with a metric for each (e.g. critical response time, wastage)	Phase 8 slide + README
7	Validation (3)	Baselines, ablations, scale test, concurrency test, multi-seed variance	Phase 7
8	TRL & demo (2)	State TRL 4 (lab-validated prototype with simulated data), a live demo, and a path to TRL 5-6 (pilot with a district control room)	Phase 8

3. Schema upgrades (new "Phase 2b" via new Alembic revisions)

Since Phases 1-2 are done, don't redo them. Add migrations:

- Spatial:** zone.geom as MultiPolygon, plus a hazard_area table (polygon, hazard_type, severity, valid_from/valid_to). Resources, requests, and shelters stay as points.
- Temporal location:** resource_location_history(resource_id, geom, recorded_at, source, confidence).
- Quantity ledger:** resource_ledger as insert-only entries (delta_qty, reason, ref_id). Derive qty_available and qty_reserved from it (a materialised view or trigger-maintained cache). Mark resources as consumable or reusable.
- Reservations with leases:** reservation(id, request_id, resource_id, qty, status, lease_expires_at, priority_snapshot).
- Pool-dependency index:** pool(id, zone_id, type_id, mobility_class) and request_pool_dependency(request_id, pool_id), kept up to date by triggers.
- Event outbox:** event_outbox(id, type, pool_id, payload, created_at, processed_at). Writing the event happens in the same transaction as the state change.
- Matching decision record:** matching_decision(request_id, resource_id, score components, final_score, algorithm_version, mode). This gives explainability and feeds the ablations.
- Lookup tables, not CHECK enums,** for resource type, plus a JSONB capabilities column (boat capacity, volunteer skills).
- Keys and metadata:** UUID primary keys, idempotency_key on requests and allocations, source_channel, and a request description.
- Audit:** audit_log with a trigger that rejects UPDATE/DELETE, REVOKE on it, and an optional hash chain (prev_hash, row_hash).

Keep the allocation state machine in the schema: PENDING -> MATCHED -> PROPOSED -> RESERVED -> DISPATCHED -> CONFIRMED -> FULFILLED, with REJECTED, EXPIRED, CANCELLED, and REASSIGNED as branches.

4. Revised phase plan

Phase 1 (done): amend the docs. Rewrite the novelty section using the 5-step format. Add the SDG mapping, TRL statement, and prior-art table.

Phase 2 (done) + 2b: the migrations above, validated against the edge cases (partial allocation, multi-resource fulfillment, multi-zone resources, stale data).

Phase 3: Core API + database-level security + events

- CRUD with pagination and filtering.
- **Security in the DB:** separate roles (``owner_role``, ``coordinator_role``, ``requester_role``) with ``GRANT``/``REVOKE``, and RLS policies. Keep JWT as the outer layer only. Mask requester contact details by role (privacy).
- Idempotency keys, outbox writing in-transaction, and a worker that consumes ``LISTEN/NOTIFY`` with the outbox as the durable fallback.

Phase 4: Innovation engine (the heart of the project)

- 1 **Candidate retrieval:** `ST_DWithin` + GiST, then filtering by pool, mobility class, and hazard reachability.
- 2 **Scoring:** weighted combination of accessibility, urgency, aging (starvation protection), pool pressure, confidence, and suitability. Keep the weights in a config table with `algorithm_version`. Offer 2-3 named modes (normal, critical-shortage, blocked-zone), but treat this as secondary. Don't oversell "adaptive weights" as the invention.
- 3 **Reservation:** one transaction doing `SELECT ... FOR UPDATE SKIP LOCKED` on candidate rows, a ledger insert, a reservation insert with a lease, and an outbox event.
- 4 **Incremental re-matching:** an event arrives, the worker looks up the dirty pool, fetches only the dependent pending requests, and re-matches them.
- 5 **Preemption rule:** only at PROPOSED/RESERVED level. After DISPATCHED, a resource is protected unless an authorized override is logged.
- 6 **Conventional baseline switch:** build `full_rescan` mode in the same codebase so the comparison is fair.

Phase 5: Scenario and ground truth

- Real-event geography and a scenario narrative, 3-5 zones, 40-60 resources, 15-20 owners, 60-100 requests over 48 hours.
- **Optimal answer key from an ILP solver** (OR-Tools/PuLP) with an explicit objective: minimise weighted unmet demand + travel cost + reallocations. Use it on the small set only.
- A **synthetic scale generator** for 10k / 100k / 1M rows, plus a hazard-change event stream (for example, a flood polygon expanding every N minutes).
- Seeded randomness and an idempotent seeder.

Phase 6: Lifecycle, audit, timeouts

- Lease expiry via ``pg_cron`` or a worker. Expiry releases units through a ledger entry and triggers the incremental re-match.
- Ranked-queue view (urgency + wait), audit read endpoints (per request, per resource, activity feed by zone and time).
- Tamper-evidence test: attempt UPDATE/DELETE as every role and show it fails, and verify the hash chain.

Phase 7: Evaluation (see section 5).

Phase 8: Dashboard and demo

- React + Leaflet: resources by status, hazard polygons, the ranked queue, an allocation detail view with its audit history and decision-score breakdown.
- **Use WebSocket or SSE, not polling**, so "real-time" is honest.
- A "simulate hazard change" button that visibly shows only the affected requests being re-matched, with a live counter of "rows evaluated: conventional vs. ResQLink". This is the best demo moment.
- Docker Compose with seed data, a README with the embedded ER diagram, and an honest "gaps" section.

Phase 9 (stretch): edge nodes with per-node bounded unit quotas and deterministic urgency-based reconciliation after reconnect. Only start it after Phases 1-8 are solid.

5. Evaluation design (3 marks, but it also underpins novelty)

Configurations (ablation):

- A: distance only
- B: + urgency
- C: + quantity/partial allocation
- D: full ResQLink
- E: D without confidence
- F: D without pool pressure
- G: D without hazard accessibility
- H: D with full re-scan instead of incremental re-matching

Metrics:

- **Database work saved:** (request, resource) pairs evaluated and rows scanned per change event, incremental vs. full re-scan.
- Re-match latency (p50/p95/p99) at 10k, 100k, 1M rows.
- Index impact: `EXPLAIN ANALYZE` with and without GiST, and sequential scan vs. index scan.
- **Double-allocations under concurrency:** target is 0 (pgbench or Locust, N parallel matchers).
- Coverage, and weighted unmet demand (critical counts more).
- Accuracy vs. the ILP optimum (optimality gap).
- Stale-resource match rate, reallocation count, and travel time via accessibility vs. straight-line.
- Fairness, meaning the wait time of low-urgency requests.
- 5+ random seeds with mean $\pm$ std, and an honest section on where the engine loses.

6. Patent strategy

Assessment: the broad idea (nearby resources + urgency, in a database) is crowded prior art and is weak. Under Section 3(k) of India's Patents Act, algorithms and software per se are excluded. The patentable angle has to be a technical solution to a technical problem with a technical effect. The claim should therefore be framed as database behaviour: *maintaining a dependency index mapping pending requests to resource pools; on a state-change event, identifying the affected pool; re-evaluating only dependent requests; and committing leased reservations atomically, thereby reducing computation and eliminating duplicate allocation.* The confidence and hazard mechanisms become dependent claims.

Steps, in order:

- 1 **Prior-art search** (Google Patents, Espacenet, Patentscope, InPASS), CPC classes G06Q 10/06, G06Q 50/26, G06F 16/23, G06F 16/27, G06F 16/29. Build a table of what each document does *not* do.
- 2 **Check the institute's IP policy** and who is listed as inventor.
- 3 **File a provisional specification before the hackathon submission or demo.** A competition submission, slide deck, or public GitHub repo is a public disclosure. A narrow grace period (Section 31) exists but shouldn't be your strategy.
- 4 Keep the repo private until filed. Keep a dated lab notebook and benchmark logs as evidence of the technical effect.

- 5 Talk to a patent attorney or your university's IPR cell before filing. Expect that the final claims may be narrower than you hope.

7. Timeline (adjust to your deadline)

Week	Work
1	Doc amendments, Phase 2b migrations, prior-art search starts
2	Phase 3 (roles, RLS, outbox)
3-4	Phase 4 (engine, reservation, incremental re-match, full-rescan baseline)
4	Phase 5 (scenario, ILP truth, scale generator)
5	Phase 6
6	Phase 7 (benchmarks, ablations, charts)
7	Phase 8, SDG/TRL slides, demo rehearsal
Before any public submission	Provisional patent filed

8. Risks

- **Scope creep:** drop Phase 9 and the adaptive-mode variations first if time runs short.
- **A weak incremental-vs-full gap at small scale:** that's why the 100k-1M synthetic set is mandatory.
- **Overclaiming:** report ablations honestly. If a component doesn't help, say so and drop it from the claims.

I can turn this into a downloadable document if you want. The most useful next step is to write the **Phase 2b Alembic migrations** and the revised **Phase 4 prompt**, since those carry the invention.

Final recommendation

Proceed with this plan. The next implementation checkpoint should be Phase 2b followed by a tightly scoped Phase 4 prototype that demonstrates the central loop: resource-state change -> affected pool detection -> dependent-request re-match -> atomic lease reservation -> durable event -> audit record. That loop is where the project earns both its DBTHON differentiation and its strongest potential IP narrative.

Important: this document is an engineering and strategy assessment, not legal advice or a patentability determination.